

7.2 While loops

Settings

Beta features

Introduction to while loops

A **while loop** runs a block of code repeatedly as long as the while loop's condition is True. If the while loop ends. Ex: A guessing game is played until a player guesses the correct answer.

Each execution of the loop is called an iteration.

Construct 7.2.1: While loop.

```
while expression: # Loop expression
    # Loop body: Statements to execute
    # if the loop expression evaluates to True

# Statements to execute after the expression evaluates to False
```

PARTICIPATION ACTIVITY

7.2.1: While loop.

Start

2x speed

```
value = 1

while value < 6:
    print(value)
    value = value * 2

print("Done")
```

Python Interpreter

8

value

Output

```
1
2
4
Done
```

Captions ^

1. First, the value variable is initialized to 1.
2. The while loop statement checks if the while loop condition is True. Here, the condition is True. Thus, the code in the loop body executes.
3. The while loop continues so long as the while loop condition is True.
4. The while loop ends when the while loop condition is False. Thus, the while loop ends when the value variable is 8.

**PARTICIPATION
ACTIVITY**

7.2.2: While loops.

1) What is the output of the following code?

```
total = 5

while total <= 25:
    print(total)
    total = total * 2
```

- ☐ 5
- ☐ 10
- ☐ 20
- ☐ 5
- ☐ 10
- ☐ 20
- ☐ 40
- ☐ 20
- ☐ 10
- ☐ 5

2) What does indentation indicate in a while loop, such as in the following code?

```
value = 1

while value <= 3:
    print(value)
    value = value + 1
```

- ☐ The end of the program
- ☐ The beginning of a new function
- ☐ The block of code that belongs to the while loop

3) How many times will the loop iterate in the following code?

```
score = 0

while score < 20:
    print(score, end=" ")
    score = score + 5
```

- ☐ 3
- ☐ 4
- ☐ 5

Sentinel values

A **sentinel value** is a value that causes a loop to end. Ex: A while loop runs until a user inputs the sentinel value. The user controls when the loop ends, thus distinguishing a loop with a sentinel value from a loop with a fixed condition.

PARTICIPATION ACTIVITY

7.2.3: Sentinel value.

Start

2x speed

```
password = "star"
user_input = input("Enter the password: ")

while user_input != password:
    print("Wrong password. Try again.")
    user_input = input("Enter the password: ")

print("Access granted.")
```

Python i

"star"

star

Output

```
Enter the password: kite
Wrong password. Try again.
Enter the password: star
Access granted.
```

Captions ^

1. The value of password, "star", is the sentinel value. The user is prompted to enter the password as user_input.
2. The while loop begins because user_input does not equal the sentinel value, "star". The user enters a wrong password and is prompted to enter the password again.
3. The user inputs the correct password, star, so the while loop ends and the print function prints "Access granted."

PARTICIPATION ACTIVITY

7.2.4: Sentinel values.

1) What is the purpose of the sentinel value in the following code?

```
sentinel = 999
input_value = 0

while input_value != sentinel:
    input_value = int(input("Enter your number or type
999 to stop: "))
```

- ☐ To track the number of inputs
- ☐ To end the loop
- ☐ To initialize the loop

2) How many times will the following loop run if the user types quit as the first input?

```
sentinel = "quit"
command = ""

while command != sentinel:
    command = input("Type quit to exit: ")
```

- ☐ 0
- ☐ 1
- ☐ 2

- 3) If the user enters 6 on the first iteration and 3 on the second iteration, what would the following output be?

```
count = 1
user_input = int(input("Enter a number 5 or greater:
"))

while user_input >= 5:
    count = count + 1
    user_input = int(input("Enter a number 5 or greater
to continue, or less than 5 to stop: "))

print("Total numbers entered: ", count)
```

- ☐ Enter a number 5 or greater: 6
Enter a number 5 or greater to continue, or less than 5 to
stop: 3
Enter a number 5 or greater to continue, or less than 5 to
stop:
Enter a number 5 or greater: 6
Enter a number 5 or greater to continue, or less than 5 to
stop: 3
Total numbers entered: 1
- ☐ Enter a number 5 or greater: 6
Enter a number 5 or greater to continue, or less than 5 to
stop: 3
Total numbers entered: 2

Infinite loops

An **infinite loop** is a loop that never stops running because the loop's condition is always True. Ex: A loop without any condition to stop.

A common error is to accidentally create an infinite loop by assuming equality will be reached. Good practice is to use less than along with equality in a loop expression to help avoid unintended infinite loops.

An infinite loop can be valid in some programs. Ex: An infinite loop in a game checks for user input in a loop.

PARTICIPATION ACTIVITY

7.2.5: Infinite loop.

Start

2x speed

```
score = 1

while score > 0:
    score = score * 2
    print(score)
```

Python interpreter

8

score

Output

2
4
8
...

Captions ^

1. First, the score variable is initialized to 1.
2. The while loop statement checks if the while loop condition is True. The condition is True greater than 0. Thus, the code in the loop body executes.
3. The while loop executes infinitely because score will always be greater than 0.

**PARTICIPATION
ACTIVITY**

7.2.6: Infinite loops.

1) Which of the following would stop the infinite loop in the previous animation?



```
score = 1
while score > 0:
    score != 1
    print(score)
```



```
score = 1
while score > 0:
    score = score + 5
    print(score)
```



```
score = 1
while score < 10:
    score = score * 2
    print(score)
```

2) What happens when the following code is run?

```
name = ""
while True:
    name += "a"
    print(name)
```



The while loop runs once, then ends.



The program enters an infinite loop.



The while loop runs but skips all indented code inside the while loop.

3) What is the final output of the following code?

```
index = 0

while index > 0:
    print(index, end=" ")
    index = index - 1
print("Bye")
```

- ☐ An infinite loop
- ☐ Bye
- ☐ Bye Bye

4) What is the output of the following code?

```
total = 10

while total != 3:
    print(total, end=" ")
    score = total / 2
```

- ☐ An infinite loop
- ☐ 10.0 5.0 2.5 1.25
- ☐ 10.0 5.0

CHALLENGE ACTIVITY

7.2.1: While loops.

763650.1722066.qx3zqy7

Start

A while loop reads floating-point numbers from input into variable `value_in`. Write an expression while `value_in` is greater than -1.0.

[Click here for example](#) ▼

Note: First, delete the comment `""" Your code goes here """`. Then, insert your expression condition.

```
1 value_in = float(input())
2 while """ Your code goes here """:
3     print(f"User entered {value_in}")
4     value_in = float(input())
5
6 print("Exit")
```

1

2

Check**Next level**

View solution ▼ *(Instructors only)*